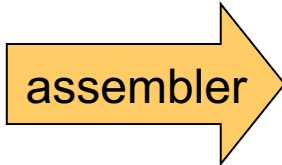


Assemblatore per il linguaggio Hack

```
@i
M=1    // i = 1
@sum
M=0    // sum = 0
(LOOP)
@i      // se i>RAM[0] goto WRITE
D=M
@R0
D=D-M
@WRITE
D;JGT
...
```



assembler

```
0000000000010000
1110111111001000
0000000000010001
1110101010001000
0000000000010000
1111110000010000
0000000000000000
1111010011010000
0000000000010010
1110001100000001
...
```

L'assembler si occupa della traduzione del linguaggio assembly (simbolico) al relativo codice in linguaggio macchina (binario).

Nel caso del linguaggio Hack, ogni istruzione in linguaggio assembly viene tradotta in un **codice a 16 bit**, ad eccezione delle direttive di dichiarazione di etichette, che semplicemente definiscono dei simboli e che non vengono quindi tradotte.

- In **input** all'assemblatore forniamo un file in linguaggio assembly Hack (estensione `.asm`)
- in **output** l'assemblatore genera un file contenente per ogni riga un numero binario a 16 bit **scritto in ASCII** (estensione `.hack`)

Nota: In un sistema reale, l'assemblatore non scriverebbe in ASCII ma in binario. Qui invece viene usato un formato testuale per permetterci comunque di aprire ed ispezionare i file codificati, e di caricarli nel CPUEmulator.

Programma assembly

```
// Computes 1+...+RAM[0]
// and stores the sum in RAM[1].
    @i
    M=1    // i = 1
    @sum
    M=0    // sum = 0
(LLOOP)
    @i     // if i>RAM[0] goto WRITE
    D=M
    @0
    D=D-M
    @WRITE
    D;JGT
    @i     // sum += i
    D=M
    @sum
    M=D+M
    @i     // i++
    M=M+1
    @LOOP  // goto LOOP
    0;JMP
(WRITE)
    @sum
    D=M
    @1
    M=D    // RAM[1] = the sum
(END)
    @END
    0;JMP
```

Programma assembly (o “**programma ISA**”) composto da:

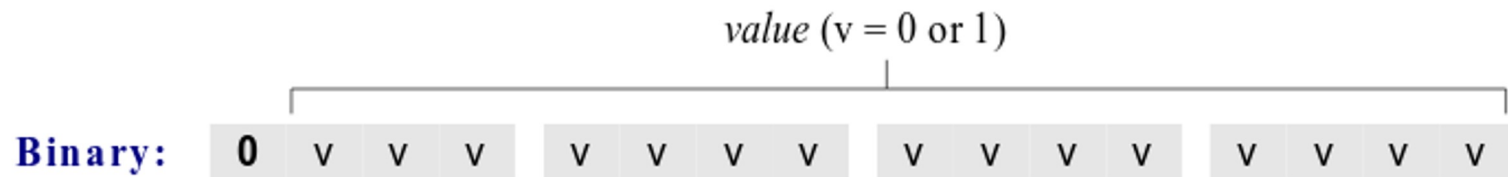
- A-instructions
- C-instructions
- dichiarazioni di etichette

Ulteriori righe possono essere commenti, spazi e righe vuoti, simboli di tabulazione ‘\t’, o simboli di fine linea come ‘\r’ o ‘\n’ :

Il nostro obiettivo (progetto 5) è programmare l’assemblatore.

Traduzione di A-instructions

Symbolic: `@value` // Where *value* is either a non-negative decimal number
 // or a symbol referring to such number.



Approccio di traduzione in codice macchina:

- Se *value* è un **numero decimale**, basta considerare per la sua sequenza “v v ... v” la rappresentazione di *value* in notazione posizionale binaria a **15 bit**: è un indirizzo, quindi non negativo!

Esempio: @3 ➔ 0000000000000011

- Se *value* è un **simbolo** (variabile o etichetta) vedremo dopo come procedure.

Traduzione di C-instructions

dest = comp; jump

comp

dest

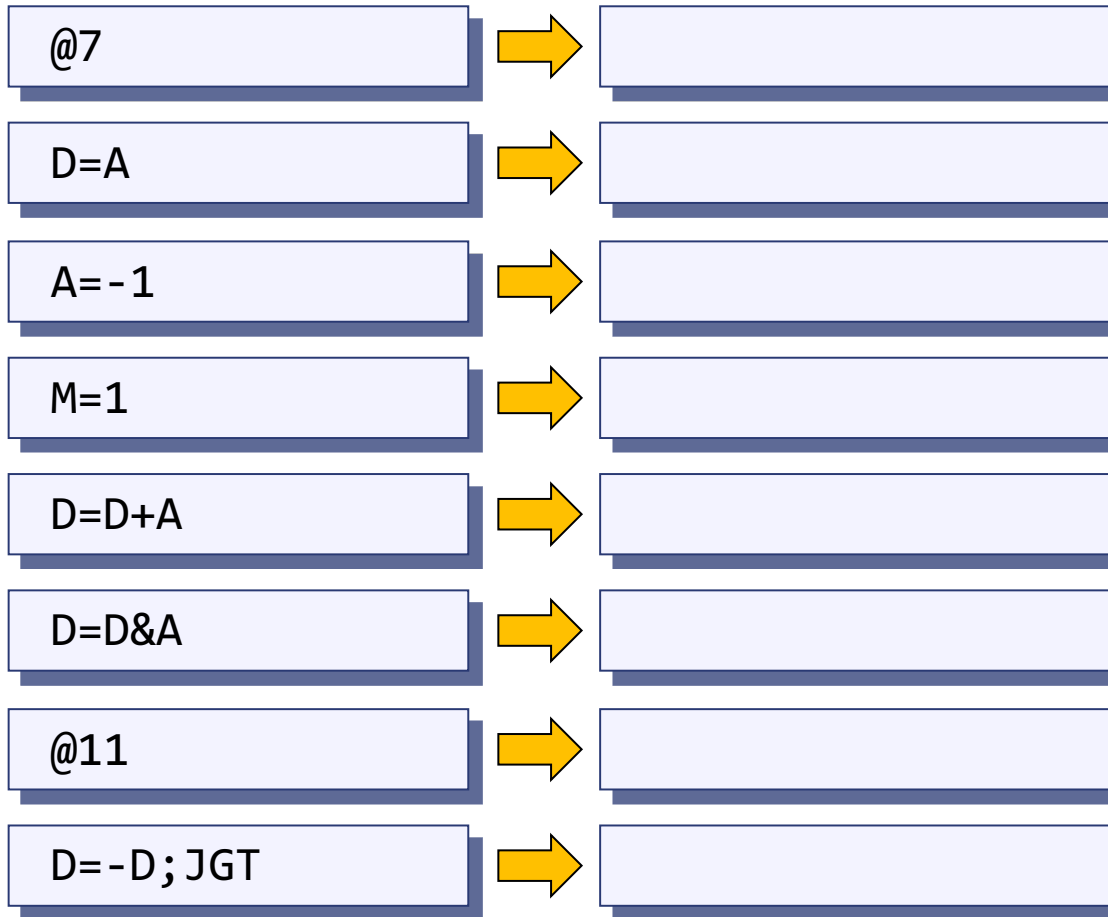
jump

binary: 1 1 1 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

(when a=0) <i>comp</i>	c1	c2	c3	c4	c5	c6	(when a=1) <i>comp</i>	d1	d2	d3	Mnemonic	Destination (where to store the computed value)
0	1	0	1	0	1	0		0	0	0	null	The value is not stored anywhere
1	1	1	1	1	1	1		0	0	1	M	Memory[A] (memory register addressed by A)
-1	1	1	1	0	1	0		0	1	0	D	D register
D	0	0	1	1	0	0		0	1	1	MD	Memory[A] and D register
A	1	1	0	0	0	0	M	1	0	0	A	A register
!D	0	0	1	1	0	1		1	0	1	AM	A register and Memory[A]
!A	1	1	0	0	0	1	!M	1	1	0	AD	A register and D register
-D	0	0	1	1	1	1		1	1	1	AMD	A register, Memory[A], and D register
-A	1	1	0	0	1	1	-M					
D+1	0	1	1	1	1	1			j1	j2	j3	
A+1	1	1	0	1	1	1	M+1	(out < 0)	(out = 0)	(out > 0)	Mnemonic	Effect
D-1	0	0	1	1	1	0		0	0	0	null	No jump
A-1	1	1	0	0	1	0	M-1	0	0	1	JGT	If out > 0 jump
D+A	0	0	0	0	1	0	D+M	0	1	0	JEQ	If out = 0 jump
D-A	0	1	0	0	1	1	D-M	0	1	1	JGE	If out ≥ 0 jump
A-D	0	0	0	1	1	1	M-D	1	0	0	JLT	If out < 0 jump
D&A	0	0	0	0	0	0	D&M	1	0	1	JNE	If out ≠ 0 jump
D A	0	1	0	1	0	1	D M	1	1	0	JLE	If out ≤ 0 jump
								1	1	1	JMP	Jump

Esempi

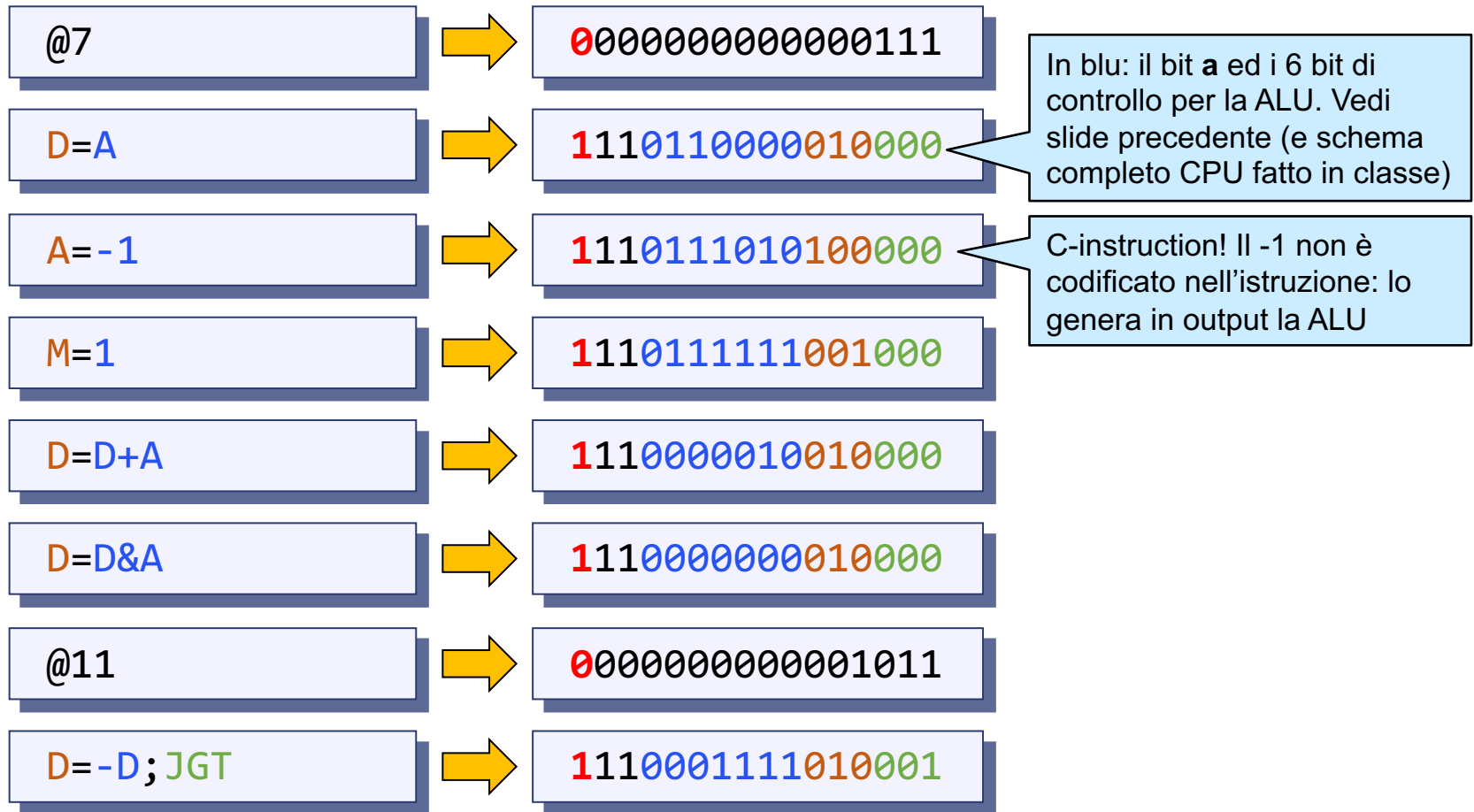
Generare il codice assembly relativo alle A/C-instructions mostrate:



Nota: questi esercizi servono per capire come funziona l'assemblatore (e quindi anche per svolgere il progetto); non è una tipologia di esame. Idem per altri esercizi in queste slide.

Esempi

Generare il codice assembly relativo alle A/C-instructions mostrate:



Nota: questi esercizi servono per capire come funziona l'assemblatore (e quindi anche per svolgere il progetto); non è una tipologia di esame. Idem per altri esercizi in queste slide.

Gestione dei simboli

I **simboli** sono usati per identificare indirizzi di istruzioni in ROM (destinazione dei salti) oppure indirizzi in memoria RAM (variabili).

Sono costituiti da questi possibili caratteri:

- Lettere minuscole (comprese fra 'a' e 'z')
- Lettere maiuscole (comprese fra 'A' e 'Z')
- Cifre decimali (comprese fra '0' e '9')
- Simboli particolari: '_', '.', '\$'

Il primo carattere di un simbolo **non** può essere una cifra decimale.

Ricordarsi di questo. In pratica: se in un programma ISA in input leggiamo una istruzione che inizia con “@” seguito da una cifra, allora sappiamo che si tratta di una normale A-instruction, altrimenti di un simbolo.

Per capire di quale simbolo si tratti (etichetta/variabile), vedremo fra poco.

Simboli predefiniti

Alcuni simboli sono **predefiniti**, ed hanno già un valore assegnato di default. Ne conosciamo già alcuni, che abbiamo imparato ad usare:

R0	0
R1	1
...	
R15	15
SCREEN	16384
KBD	24576

Altri sono sempre predefiniti, ma non abbiamo ancora visto a cosa servono (si tratta di puntatori di controllo della Virtual Machine, che vedremo dopo):

SP	0
LCL	1
ARG	2
THIS	3
THAT	4

Simboli predefiniti

Registri virtuali:

I simboli R0,..., R15 sono automaticamente predefiniti per far riferimento agli indirizzi di memoria RAM 0,...,15

Puntatori per I/O:

I simboli SCREEN and KBD fanno riferimento agli indirizzi RAM 16384 e 24576, rispettivamente (indirizzi base della mappa in memoria dello schermo, e della tastiera)

Puntatori di controllo della Virtual Machine:

I simboli SP, LCL, ARG, THIS, and THAT sono automaticamente predefiniti per far riferimento agli indirizzi di RAM 0, 1, 2, 3, e 4, rispettivamente

```
@R0
D=M
@END
D;JLE
@counter
M=D
@SCREEN
D=A
@X
M=D
(L O O P )
@X
A=M
M=-1
@X
D=M
@32
D=D+A
@X
M=D
@counter
MD=M-1
@LOOP
D;JGT
(E N D )
@END
0;JMP
```

Simboli non predefiniti

Etichette:

Usate per identificare i punti di destinazione delle istruzioni di salto (vedi LOOP e END nell'esempio). Definite tramite la direttiva (YYY) che definisce l'etichetta YYY a cui verrà assegnato come valore l'indirizzo di memoria ROM in cui verrà caricata la prima istruzione successiva alla direttiva (YYY)

Variabili:

Usate nelle A-instruction per identificare celle di memoria RAM da dedicare a specifiche variabili (vedi counter e x nell'esempio). Riceveranno un valore che va dal 16 in poi. I valori vengono assegnati seguendo l'ordine in cui tali variabili appaiono. Nell'esempio counter varrà 16, mentre x varrà 17.

```
@R0
D=M
@END
D;JLE
@counter
M=D
@SCREEN
D=A
@x
M=D
(LOOP)
@x
A=M
M=-1
@x
D=M
@32
D=D+A
@x
M=D
@counter
MD=M-1
@LOOP
D;JGT
(END)
@END
0;JMP
```

Symbol table

Per la gestione e traduzione dei simboli, creiamo una **tabella dei simboli** (*symbol table*), che contiene per ogni simbolo (predefinito o no) il valore corrispondente. Quanto detto finora comporta che:

- Quando creiamo la symbol table, possiamo direttamente inserire tutti i simboli predefiniti con i loro rispettivi valori;

R0	0
R1	1
R2	2
...	...
R15	15
SCREEN	16384
KBD	24576
SP	0
LCL	1
ARG	2
THIS	3
THAT	4

- Altri simboli, incontrati durante la scansione del programma in questione, verranno aggiunti man mano, calcolandone il valore.

Il processo di assemblaggio: procedura

Inizializzazione:

- Apertura in lettura del file `.asm` in input.
- Inizializzazione della symbol table con i simboli predefiniti

Prima passata:

Si scorre l'input per inserire in symbol table le etichette incontrate, tenendo un contatore del numero di **A/C-instruction** viste finora (iniziando dalla numero 0). Quando si incontra una **definizione** di etichetta le si assegna il valore di tale contatore (più 1) in quel momento (e non si incrementa il contatore: non è una A/C-instruction!).

Seconda passata:

Si apre in scrittura il file di output `.hack` e si scorre di nuovo l'input:

- per A- e C-instructions si scrive in output il relativo codice. In particolare:
- se una A-instruction usa un simbolo (vedi prima su come riconoscerli): si cerca il valore in symbol table; se il simbolo è assente (quindi **non** è una etichetta, ma una variabile) si assegna un valore progressivo, partendo da 16, e lo si memorizza in symbol table. Se presente (o se appena inserito), si usa questo valore per generare in output il codice dell'instruction.

Esempio: inizializzazione

```
@i
M=1    // i = 1
@sum
M=0    // sum = 0
(LOOP)
@i     // se i>RAM[0] goto WRITE
D=M
@R0
D=D-M
@WRITE
D;JGT
@i     // sum += I
D=M
@sum
M=D+M
@i     // i++
M=M+1
@LOOP // ricomincia ciclo
0;JMP
(WRITE)
@sum
D=M
@R1
M=D    // RAM[1] = sum
(END)
@END
0;JMP
```

Symbol table

R0	0
R1	1
R2	2
...	...
R15	15
SCREEN	16384
KBD	24576
SP	0
LCL	1
ARG	2
THIS	3
THAT	4

(Il programma calcola la somma
1+...+RAM[0], scrivendo il totale in RAM[1])

Esempio: prima passata

```
0    @i
1    M=1    // i = 1
2    @sum
3    M=0    // sum = 0
      (LOOP)
4    @i    // se i>RAM[0] goto WRITE
5    D=M
6    @R0
7    D=D-M
8    @WRITE
9    D;JGT
10   @i    // sum += I
11   D=M
12   @sum
13   M=D+M
14   @i    // i++
15   M=M+1
16   @LOOP // ricomincia ciclo
17   0;JMP
      (WRITE)
18   @sum
19   D=M
20   @R1
21   M=D    // RAM[1] = sum
      (END)
22   @END
23   0;JMP
```

Symbol table

R0	0
R1	1
R2	2
...	...
R15	15
SCREEN	16384
KBD	24576
SP	0
LCL	1
ARG	2
THIS	3
THAT	4
LOOP	4
WRITE	18
END	22
i	16
sum	17

Se ci fossero righe di commento nel codice, queste sarebbero ignorate (non contate).

Esempio: seconda passata

```
@i
M=1    // i = 1
@sum
M=0    // sum = 0
(LOOP)
@i     // if i>RAM[0] goto WRITE
D=M
@R0
D=D-M
@WRITE
D;JGT
@i     // sum += I
D=M
@sum
M=D+M
@i     // i++
M=M+1
@LOOP  // ricomincia ciclo
0;JMP
(WRITE)
@sum
D=M
@R1
M=D    // RAM[1] = sum
(END)
@END
0;JMP
```

assemblaggio

per esempio, @i è
codificato come 16

per esempio, il salto a
LOOP è ora a ROM[4]

Codice macchina

```
0000000000010000
1110111111001000
0000000000010001
1110101010001000
0000000000010000
1111110000010000
0000000000000000
1111010011010000
0000000000010010
1110001100000001
0000000000010000
1111110000010000
0000000000010001
1111000010001000
0000000000010000
1111110111001000
0000000000000100
1110101010000111
0000000000010001
1111110000010000
0000000000000001
1110001100001000
0000000000010110
1110101010000111
```

Nota: solo le righe con A/C-instructions
generano un relativo codice macchina

Esercizi

Completare la symbol table per questi due frammenti codice (**solo simboli non predefiniti**):

```
0    @x
1    M=1
2    @y
    (CICLO)
3    @1
4    D=M
5    @x
6    D=D-M
7    @END
8    D;JLE
9    @CICLO
10   0;JMP
    (END)
11   @END
12   0;JMP
```

...

```
0    @R1
1    D=M
2    @from
3    M=D
4    @i
5    M=D
6    @R2
7    D=M
8    @to
9    M=D
    (WHILE)
10   @to
11   D=M
12   @i
13   D=D-M
14   @END
15   D;JLT
16   @i
17   A=M
18   D=M
```

```
19   @5
20   D=D-A
21   @SKIP
22   D;JLE
23   @count
24   M=M+1
    (SKIP)
25   @WHILE
26   0;JMP
    (END)
27   @END
28   0;JMP
```

...

Esercizi: soluzione

Completare la symbol table per questi due frammenti codice (**solo simboli non predefiniti**):

```
0    @x
1    M=1
2    @y
    (CICLO)
3    @1
4    D=M
5    @x
6    D=D-M
7    @END
8    D;JLE
9    @CICLO
10   0;JMP
    (END)
11   @END
12   0;JMP
```

```
...
CICLO    3
END      11
x        16
y        17
```

```
0    @R1
1    D=M
2    @from
3    M=D
4    @i
5    M=D
6    @R2
7    D=M
8    @to
9    M=D
    (WHILE)
10   @to
11   D=M
12   @i
13   D=D-M
14   @END
15   D;JLT
16   @i
17   A=M
18   D=M
```

```
19   @5
20   D=D-A
21   @SKIP
22   D;JLE
23   @count
24   M=M+1
    (SKIP)
25   @WHILE
26   0;JMP
    (END)
27   @END
28   0;JMP
```

```
...
WHILE    10
END      27
SKIP     25
from     16
i        17
to       18
count    19
```

Qualche consiglio per il progetto 5

Procedere per fasi:

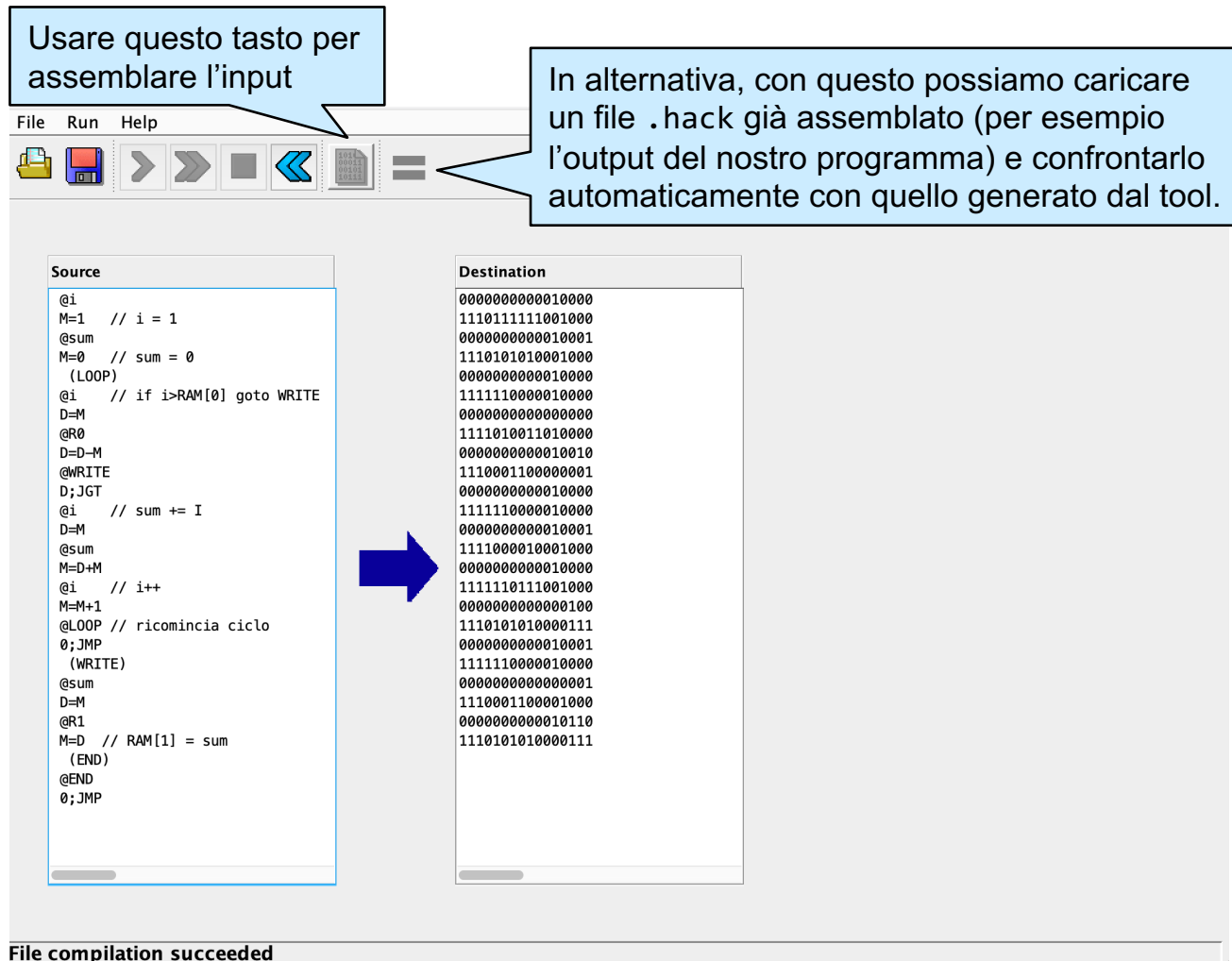
- Implementare prima un assembler che non considera i simboli (quindi senza alcuna symbol table, usando una sola passata). Utilizzare come base di partenza uno degli esempi di programmazione C che vedremo.
Deve generare un file con il medesimo nome dell'input ma con l'estensione `.asm` sostituita da `.hack`
Controllare la correttezza usando come prova dei programmi in assembly che non utilizzino simboli
- Passare poi all'implementazione completa: Implementare (separatamente) la struttura dati symbol table. Deve fornire funzionalità di base:
(1) inserimento di nuovi simboli (col relativo valore) e
(2) recupero del valore, dato un simbolo.
- Estendere l'implementazione precedente per fare le due passate come da slide precedenti. Controllare correttezza (`Pong.asm` come esempio non banale) usando il programma `Assembler.sh`

Tool Assembler

Possiamo usare l'applicazione Assembler fornita (Assembler.sh) per testare i file .hack in linguaggio macchina generati dal nostro codice di progetto.

Usare questo tasto per assemblare l'input

In alternativa, con questo possiamo caricare un file .hack già assemblato (per esempio l'output del nostro programma) e confrontarlo automaticamente con quello generato dal tool.



The screenshot shows the Assembler tool interface. The top menu bar includes 'File', 'Run', and 'Help'. Below the menu is a toolbar with icons for file operations (open, save, delete, copy, paste) and a button with an equals sign. The main window is divided into two panes: 'Source' on the left and 'Destination' on the right. A large blue arrow points from the Source pane to the Destination pane. The Source pane contains assembly code, and the Destination pane shows the corresponding binary output. At the bottom, a status bar indicates 'File compilation succeeded'.

```
Source
@i
M=1 // i = 1
@sum
M=0 // sum = 0
(L00P)
@i // if i>RAM[0] goto WRITE
D=M
@R0
D=D-M
@WRITE
D;JGT
@i // sum += I
D=M
@sum
M=D+M
@i // i++
M=M+1
@LOOP // ricomincia ciclo
0;JMP
(WRITE)
@sum
D=M
@R1
M=D // RAM[1] = sum
(END)
@END
0;JMP

Destination
000000000010000
1110111111001000
000000000010001
11101010001000
000000000010000
1111110000010000
000000000000000
1111010011010000
000000000010010
1110001100000001
0000000000010000
1111110000010000
0000000000010001
1111000010001000
0000000000010000
111110111001000
000000000000100
1110101010000111
0000000000010001
1111110000010000
0000000000000001
1110001100001000
0000000000010110
11101010000111
```

File compilation succeeded